

Technical Documentation – Milestone 2

Multi-Agent Diagnostic Layer

AI Smart Bug Analyzer and Fix Advisor (BugSense AI)

Prepared By: Harshitha Bejjipuram

Internship: Infosys Springboard Internship

1. Introduction

The AI Smart Bug Analyzer and Fix Advisor (BugSense AI) is an intelligent software engineering platform developed to automate software bug diagnosis using Artificial Intelligence and modern software engineering technologies. As software applications become increasingly complex, developers receive a large number of bug reports during development and production. Analyzing these reports manually requires significant effort to understand bug descriptions, examine stack traces, identify affected modules, determine severity levels, assign priorities, and locate the root cause of failures. This manual process is time-consuming, inconsistent, and often delays software maintenance and product delivery.

To address these challenges, Milestone 2 introduces a Multi-Agent Diagnostic Layer that extends the Retrieval-Augmented Generation (RAG) capabilities implemented in Milestone 1. Instead of depending on a single AI model, the platform employs multiple specialized AI agents that work together to perform different diagnostic tasks. This modular architecture improves scalability, maintainability, accuracy, and flexibility while simplifying future enhancements.

The diagnostic process begins when a user submits a bug report containing the bug title, description, runtime logs, and stack traces. The request is received through the FastAPI backend and executed using a LangGraph workflow. The Triage Agent analyzes the textual information to determine the affected software component, estimate the severity level, assign the appropriate development priority, calculate a confidence score, and provide reasoning for its predictions. The Log Analysis Agent then processes runtime exception logs and stack traces using AI-assisted analysis and regular expression pattern matching to identify exception types, failure points, source files, functions, execution paths, and possible root causes. Finally, both agent outputs are combined into a structured JSON diagnostic report that can be consumed by the frontend and backend services.

To ensure reliable communication between all system components, Pydantic models are used for request validation and response serialization. The system also supports both Google Gemini AI and offline heuristic-based logic, ensuring uninterrupted functionality even when external AI services are unavailable. By automating bug classification and log analysis, Milestone 2

2. Objectives

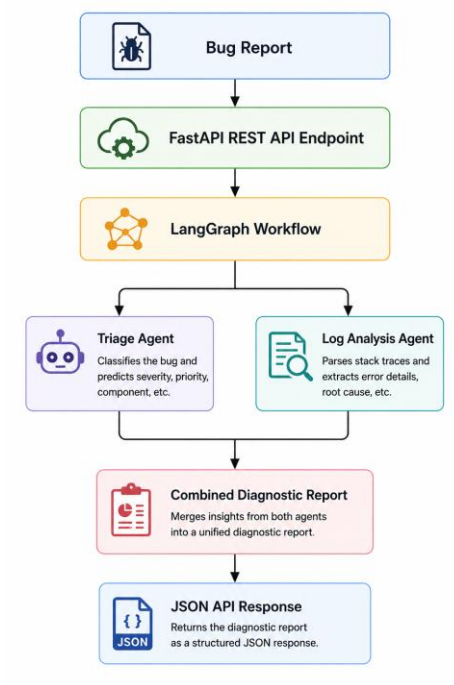
The primary objective of Milestone 2 is to develop an intelligent Multi-Agent Diagnostic System capable of automatically analyzing software bug reports and generating accurate, structured diagnostic information to support developers during software maintenance. The milestone focuses on improving debugging efficiency through AI-driven automation, workflow orchestration, and intelligent log analysis.

The specific objectives of this milestone are:

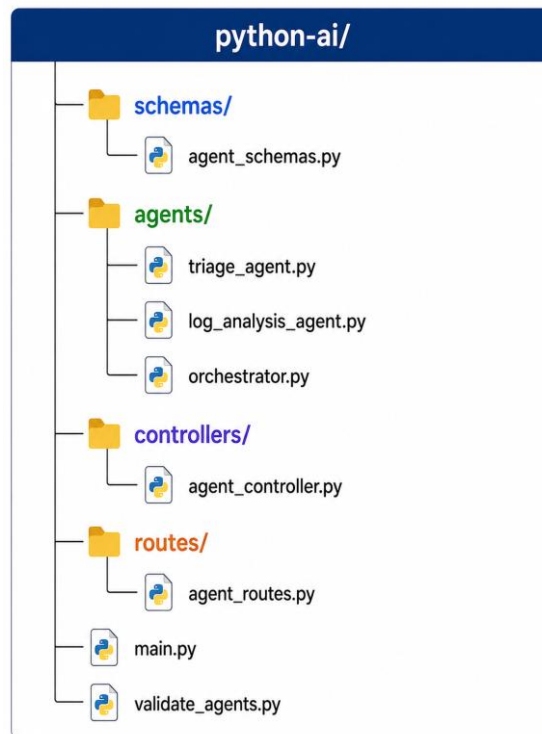
- Design and implement a modular Multi-Agent architecture for automated software bug diagnosis.
- Develop a Triage Agent capable of identifying the affected component, severity level, development priority, confidence score, and reasoning.
- Implement a Log Analysis Agent to analyze runtime logs and stack traces, detect exception types, identify failure points, locate source files and functions, and determine possible root causes.
- Integrate multiple AI agents using LangGraph to execute a sequential workflow and generate a unified diagnostic report.
- Use Pydantic models for secure request validation and structured API responses.
- Expose diagnostic services through FastAPI REST APIs for seamless integration with the existing application.
- Support both Google Gemini AI and offline heuristic-based analysis to ensure reliable execution under different environments.
- Validate the performance of the diagnostic system using automated test cases covering Authentication, Database, Backend, Frontend, API, Security, and Network modules.
- Reduce manual debugging effort, improve bug prioritization, and increase the accuracy of software fault diagnosis.
- Establish a scalable foundation for future enhancements such as AI-powered fix recommendations, automated root cause analysis, and intelligent software maintenance.

.

3. System Architecture



4. Project Directory Structure



5. Agent Architecture

Milestone 2 follows a Multi-Agent Architecture.

Each AI agent performs a dedicated responsibility.

Agent	Responsibility
Triage Agent	Bug classification
Log Analysis Agent	Stack trace parsing
LangGraph	Workflow orchestration
FastAPI	REST APIs
Controller	Business Logic

6. Triage Agent

Purpose

Automatically classify incoming software bugs.

The Triage Agent predicts

- Component
- Severity
- Priority
- Confidence
- Reasoning

Input

- Bug Title
- Bug Description
- Environment

Output

- Severity
- Priority

- Component
- Confidence Score

Decision Logic

The agent first checks whether a Gemini API key is available.

If available

↓

Uses LangChain Structured Output

Else

↓

Uses Offline Rule-Based Classification

Keyword mapping is used for

- Authentication
- Database
- API
- Backend
- Frontend
- Network
- Security

7. Log Analysis Agent

Purpose

Automatically parse application stack traces.

Supported Languages

- Python
- Java
- JavaScript

Extracted Information

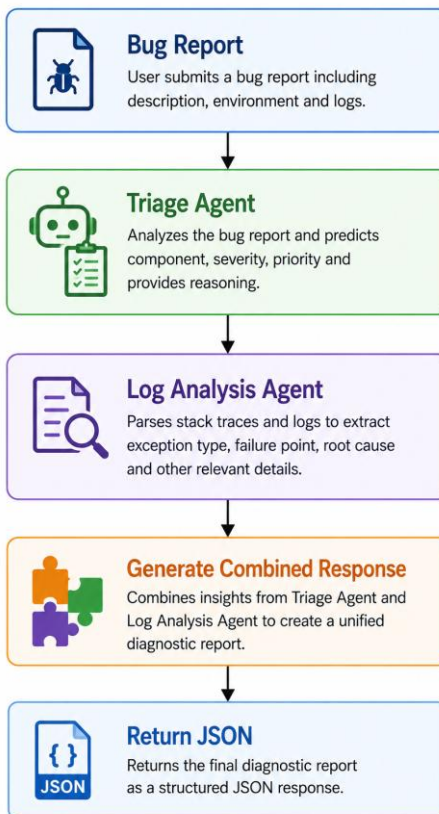
- Exception Type
- Failure Point
- File Name
- Function Name
- Line Number
- Root Cause
- Execution Path

Regex patterns are used when LLM services are unavailable.

8. Multi-Agent Orchestration

LangGraph connects multiple AI agents together.

Workflow



Benefits

- Modular Design
- Independent Agents
- Easy Future Expansion
- Better Maintainability

9. REST API Endpoints

Method	Endpoint	Description
POST	/agent/triage	Bug Classification
POST	/agent/log_analysis	Stack Trace Parsing
POST	/agent/orchestrate	Execute Full AI Workflow

10. Validation Strategy

The project includes automated validation using `validate_agents.py`

The validation verifies

- Severity Prediction
- Priority Prediction
- Exception Detection
- Failure Point Detection

Sample Test Categories

- Authentication
- Backend
- Database
- API
- Security
- Network

11. Technologies Used

TECHNOLOGY STACK		
 Layer	 Technology	
 Programming		Python
 Framework		FastAPI
 AI Framework		LangChain
 Workflow Engine		LangGraph
 Validation		Pydantic
 LLM		Google Gemini API
 Pattern Matching		Regular Expressions

12. Results

The implementation successfully achieved

IMPLEMENTATION STATUS		
Feature		Status
 Triage Agent		 Completed
 Log Analysis Agent		 Completed
 LangGraph Workflow		 Completed
 REST APIs		 Completed
 Structured Response		 Completed
 Validation		 Completed

Validation Accuracy

- Severity Prediction → 100%
- Priority Prediction → 100%
- Exception Detection → 100%
- Failure Point Detection → 100%

13. Future Enhancements

Future milestones will include

- Retrieval-Augmented Generation (RAG)
- AI Fix Recommendation Engine
- Root Cause Knowledge Base
- Automatic Code Suggestions
- Developer Chat Assistant
- Bug Resolution Prediction

14. Conclusion

Milestone 2 successfully introduces a robust Multi-Agent Diagnostic Layer into the AI Smart Bug Analyzer platform. By combining a Triage Agent, Log Analysis Agent, and LangGraph orchestration, the system automates software bug classification and runtime error analysis. The modular design improves scalability, maintainability, and diagnostic accuracy while providing structured API responses that support future AI-powered debugging features.